

1.4 Prompt Engineering

把提示词做成 Agent 的行为契约

从「写提示词」到「写行为契约」——定义职责与边界、校准输出与过程、实现复用、安全与版本治理。

系统规则 · System

本次任务 · User

不可信数据 · Data

Prompt = 可执行的行为契约



Prompt Engineering：把提示词做成行为契约

核心结论

Prompt Engineering 不是寻找「神奇提示词」，而是把业务目标、行为规则和安全边界，写成模型可执行的行为契约。

01

定义职责与边界

明确 Agent 负责什么、不负责什么，以及信息不足时如何处置。

02

校准输出与过程

用 Few-shot 与任务拆解，让模型的行为与结果可预期、可检查。

03

复用 · 安全 · 版本

把 Prompt 模板化、防注入、纳入版本管理，像工程资产一样维护。



学完本节，你应该能够

核心结论

从「会写提示词」到「会做提示词工程」——下面六个能力，是区分业余与专业的分水岭。

01

定义 System Prompt

用 System Prompt 定义角色、目标、边界和输出风格。

02

运用 Few-shot

用范例校准格式敏感和边界模糊的任务。

03

任务拆解

将复杂任务拆成可检查的步骤。

04

模板化

将 Prompt 模板化，分离固定规则与运行时数据。

05

识别注入

识别直接与间接 Prompt 注入。

06

版本治理

建立版本号、变更记录和最小回归测试。



模型自己会写 Prompt，为什么还要人来学？

核心结论

模型可以生成措辞，但不能替人做业务决策和工程决策。

模型可以协助

可协助

• 优化表达、补充示例

• 生成 Prompt 初稿

• 建议输出格式

• 改写任务描述

工程师必须负责

须负责

✓ 定义 Agent 的职责和目标

✓ 决定行为优先级与失败策略

✓ 判断哪些风险必须程序拦截

✓ 制定验收标准并承担结果责任

Prompt 影响模型的行为倾向；程序与 Harness 保证系统不能越界。

先定义岗位，再交代任务



核心结论

System Prompt 是 Agent 的「岗位说明书」，负责定义跨任务保持不变的规则。

System Prompt

长期 · 不变

定义长期角色与目标

规定稳定的行为边界

跨任务保持一致

User Prompt

本次 · 变化

✓

描述本次具体任务

✓

提供本次输入与要求

✓

随请求发生变化

一个够用的 System Prompt 包含： 角色 · 目标 · 边界 · 流程 · 输出 工程师必须确认：Agent 做什么、先做什么、不能做什么、信息不足怎么办。

角色不是头衔，而是行为



核心结论

「资深专家」无法测试；可观察的行为才是有效约束。

弱角色定义

不可测试

×

你是一名资深专家

×

请认真、深入分析

×

给出专业建议

可验证的行为定义

可观察

✓

先检查正确性与安全性

✓

每个结论引用具体证据

✓

信息不足时明确说明，不猜测

行为约束要回答四个问题： 优先级是什么？ · 判断依据是什么？ · 失败时怎么办？ · 哪些事情不能做？



让结果适合使用场景

核心结论

内容正确不等于产品可用；表达方式必须匹配实际使用场景。

普通用户助手	少术语，先结论再解释
开发者 CLI	短句、状态明确、避免铺垫
代码审查页面	风险等级、代码位置、修改建议

- 详细程度
- 语气
- 结构

自然语言可以约束表达；下游程序需要稳定字段时，应交给 Structured Output。



用范例校准模型

核心结论

当格式敏感或判断边界难以描述时，2~3 个高质量示例通常比更多抽象规则更有效。

①

典型正确样例

展示标准输入与输出，建立「合格长什么样」的基准。

②

边界样例

说明容易混淆的判断，划清行为分界的灰度区。

③

信息不足 / 拒绝样例

说明何时不应强行回答，明确放弃的边界。

工程师必须挑选能代表真实任务分布的样例；偏斜或错误示例会放大误判。Few-shot 负责校准行为，不负责授予权限。



不要让模型一次完成所有事情

核心结论

复杂 Prompt 越长不一定越稳定；真正的问题往往是任务存在步骤依赖。

一次完成所有任务

易失控

×

容易跳过分析，直接给答案

×

过程不可检查

×

错误来源难定位

×

容易声称未实际完成的动作

分步完成任务

可验证

✓

每一步只有一个清晰目标

✓

中间结果可以验证

✓

可以定位问题出现在哪一步

✓

后一步基于真实的前一步结果

拆解原则：前一步的输出，必须成为后一步的有效输入。



分析 — 执行 — 检查

核心结论

入门阶段先掌握三段式拆解，不必立即扩展成完整 Agent Loop。



适合拆解：多目标、步骤有依赖、依赖外部信息、错误成本高。 无需拆解：简单分类、短文本改写、单一字段提取。



把一次性文本变成可复用组件

核心结论

Prompt 模板的本质，是把固定规则与可变数据分开。

固定规则

● 角色与目标

● 行为流程与边界

● 输出要求

● Prompt 名称与版本

运行时数据

● 用户任务

● 代码、网页、文档

● 工具结果与上下文

● 每次请求的变量

模板化解决：重复副本、漏改规则、变量缺失、指令与数据混写。Prompt 应像 API 一样维护：有明确契约、参数要求和版本号。



工程师要设计什么？

核心结论

模板不是简单的字符串拼接，而是 Prompt 的最小工程接口。

- 01

模板名称和当前版本
- 02

必填变量及其含义
- 03

变量长度与内容边界
- 04

固定规则与数据区域
- 05

缺失变量时的失败方式

标签可以帮助模型区分指令和数据，但不能代替安全控制。



区分指令和不可信数据

核心结论

凡是来自用户、网页、文件、邮件或工具的文本，都不能因为进入上下文就自动成为可信指令。

直接 Prompt 注入

直

直 指令来源：用户直接输入恶意指令

直 典型场景：要求忽略规则、泄露 System Prompt

间接 Prompt 注入

间

间 指令来源：Agent 读取的外部内容

间 典型场景：网页、邮件、PDF、代码注释藏有恶意指令

两者都必须识别，因为它们最终都会以文本形式进入模型上下文。



三道基础防线

核心结论

防注入不能只靠提醒模型，也不能只靠关键词过滤。

1

分离指令与数据

稳定规则放在 System Prompt；外部内容放入数据区域。

2

声明信任边界

明确外部文本仅用于分析，不能覆盖系统规则。

3

程序拦截高风险动作

外发、写入、命令执行必须经过权限与人工确认。

Prompt 告诉模型「不要越界」；Harness 保证即使模型判断失败，也不能直接执行危险动作。



知道线上到底运行的是哪份规则

核心结论

Prompt 会改变 Agent 行为，因此必须像代码一样可追踪、可验证、可回退。

01 当前运行哪个 Prompt 版本？

02 这个版本修改了什么，为什么修改？

03 哪些请求使用了这个版本？

04 修改后，原有典型任务是否仍然正常？

不要使用 final_v3 式命名；版本号必须对应明确的行为变化。



版本与最小回归

核心结论

每次 Prompt 变化，都应有变更记录和最小回归测试。

补丁版本	文案微调，语义基本不变 → 1.0.0 → 1.0.1
次版本	增加规则或示例，行为可预期变化 → 1.0.0 → 1.1.0
主版本	角色、目标或输出协议明显改变 → 1.0.0 → 2.0.0

最小回归至少包含：正常样例、边界样例、注入样例。调用日志应记录 Prompt 名称、版本、模型与执行结果。



Agent 开发工程师真正需要完成什么？

核心结论

模型负责生成与执行；工程师负责定义、取舍、约束和验收。

定义职责	明确角色、目标和优先级
定义边界	规定允许、禁止和失败处理
定义合格结果	设计输出风格与 Few-shot
设计任务步骤	识别依赖，拆成分析、执行、检查
建立工程管理	模板化、防注入、版本化、回归测试

Prompt Engineering 的最终交付物，是一份可复用、可检查、可演进的行为契约。



把一句话改造成工程化 Prompt

核心结论

将「你是一个厉害的代码审查专家，请认真审查用户代码」改造成工程化 Prompt。

- 01 补充角色、目标、行为边界和输出风格
- 02 增加正常样例与「无明确问题」样例
- 03 将任务拆成「分析—审查—检查」
- 04 分离固定规则与运行时变量
- 05 加入间接 Prompt 注入防护

验收结论：设置名称、版本号、变更说明，不可信内容不升级为指令，高风险行为由程序或 Harness 控制。

06

- 本节收尾

从 Prompt 到 Structured Output

本节结论

Prompt 解决「如何让模型按规则工作」。

下一节问题

即使模型理解了规则，程序怎样稳定、可靠地读取它的输出？

→ **下一节：Structured Output 结构化输出**
把模型的「理解」变成程序可以稳定消费的数据。